Capstone Project : E-commerce Data Analysis Project

ELiud Mule

2026-05-27

Set Working Directory

```
  setwd("C:/Users/user/Desktop/DATA SCIENTIST MULE/R/PROJECTS")
```

Phase 1 : Data Collection and Loading

Load Necessary Libraries

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(rio)
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(class)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(e1071)
```

```
##
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:ggplot2':
##
##     element
```

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ lubridate 1.9.5     ✔ tibble    3.3.1
## ✔ purrr     1.2.1     ✔ tidyr     1.3.2
## ✔ readr     2.2.0
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ randomForest::combine() masks dplyr::combine()
## ✖ e1071::element()        masks ggplot2::element()
## ✖ dplyr::filter()         masks stats::filter()
```

```
## ✖ dplyr::lag()            masks stats::lag()
## ✖ purrr::lift()           masks caret::lift()
## ✖ randomForest::margin()  masks ggplot2::margin()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
# Load the Dataset
data <- read.csv("ecommerce_data (3).csv")
head(data)
```

```
##   Customer_ID Age Gender       Location Product_Category Product_Price Quantity
## 1    CUST0001  56   Male    Christyview      Electronics        167.79        2
## 2    CUST0002  21 Female   Wolfeborough           Beauty         62.80        5
## 3    CUST0003  58   Male   Port Jessica           Sports        461.59        5
## 4    CUST0004  56 Female  Gonzalezmouth            Books        284.50        1
## 5    CUST0005  37   Male    Reillyhaven          Fashion        166.87        5
## 6    CUST0006  22   Male Lake Dianefort             Home        408.60        5
##   Order_Date   Payment_Method Discount Shipping_Cost Purchase_Amount
## 1  6/22/2025           PayPal    28.74         19.65          326.49
## 2   8/2/2025           M-Pesa     7.38         28.35          334.97
## 3 11/28/2025 Cash on Delivery    90.84         29.16         2246.27
## 4   5/3/2026      Credit Card    23.44         18.34          279.40
## 5   3/9/2026           M-Pesa    11.46         14.31          837.20
## 6  2/22/2025 Cash on Delivery    76.63         22.95         1989.32
##   Customer_Rating Order_Status
## 1               3     Returned
## 2               4    Completed
## 3               4     Returned
## 4               2     Returned
## 5               5      Pending
## 6               1     Canceled
```

Data Structure and Summary Statistics

```
str(data)
```

```
## 'data.frame':    1000 obs. of  14 variables:
##  $ Customer_ID     : chr  "CUST0001" "CUST0002" "CUST0003" "CUST0004" ...
##  $ Age             : int  56 21 58 56 37 22 69 63 50 47 ...
##  $ Gender          : chr  "Male" "Female" "Male" "Female" ...
##  $ Location        : chr  "Christyview" "Wolfeborough" "Port Jessica" "Gonzalezmout
h" ...
##  $ Product_Category: chr  "Electronics" "Beauty" "Sports" "Books" ...
##  $ Product_Price   : num  167.8 62.8 461.6 284.5 166.9 ...
##  $ Quantity        : int  2 5 5 1 5 5 3 3 1 2 ...
##  $ Order_Date      : chr  "6/22/2025" "8/2/2025" "11/28/2025" "5/3/2026" ...
##  $ Payment_Method  : chr  "PayPal" "M-Pesa" "Cash on Delivery" "Credit Card" ...
##  $ Discount        : num  28.74 7.38 90.84 23.44 11.46 ...
##  $ Shipping_Cost   : num  19.6 28.4 29.2 18.3 14.3 ...
##  $ Purchase_Amount : num  326 335 2246 279 837 ...
```

```
##  $ Customer_Rating : int  3 4 4 2 5 1 1 1 4 4 ...
##  $ Order_Status    : chr  "Returned" "Completed" "Returned" "Returned" ...
```

```
summary(data)
```

```
##  Customer_ID            Age           Gender            Location
##  Length:1000        Min.   :18.00   Length:1000        Length:1000
##  Class :character   1st Qu.:32.00   Class :character   Class :character
##  Mode  :character   Median :45.00   Mode  :character   Mode  :character
##                     Mean   :44.64
##                     3rd Qu.:58.00
##                     Max.   :70.00
##  Product_Category   Product_Price       Quantity      Order_Date
##  Length:1000        Min.   :  5.48   Min.   :1.000   Length:1000
##  Class :character   1st Qu.:128.72   1st Qu.:2.000   Class :character
##  Mode  :character   Median :257.67   Median :3.000   Mode  :character
##                     Mean   :252.72   Mean   :2.993
##                     3rd Qu.:375.93   3rd Qu.:4.000
##                     Max.   :499.52   Max.   :5.000
##  Payment_Method        Discount      Shipping_Cost    Purchase_Amount
##  Length:1000        Min.   :  0.02   Min.   : 2.040   Min.   :  17.89
##  Class :character   1st Qu.: 11.01   1st Qu.: 9.088   1st Qu.: 280.12
##  Mode  :character   Median : 31.07   Median :15.385   Median : 580.49
##                     Mean   : 39.27   Mean   :15.922   Mean   : 737.73
##                     3rd Qu.: 62.56   3rd Qu.:23.355   3rd Qu.:1094.34
##                     Max.   :147.86   Max.   :29.990   Max.   :2412.09
##  Customer_Rating Order_Status
##  Min.   :1.000   Length:1000
##  1st Qu.:2.000   Class :character
##  Median :3.000   Mode  :character
##  Mean   :2.996
##  3rd Qu.:4.000
##  Max.   :5.000
```

> The dataset contains **1,000 observations and 14 variables** covering customer demographics (Age, Gender), transaction details (Purchase_Amount, Product_Price, Quantity, Discount, Shipping_Cost), product and payment information (Product_Category, Payment_Method), order metadata (Order_Date, Order_Status, Location), and feedback (Customer_Rating). Most numeric variables have plausible ranges, and no immediate structural anomalies are apparent from the summary.

Checking for Missing Values and Duplicates

```
# Missing values per column
missing_summary <- data.frame(
  column       = names(data),
  missing_count = colSums(is.na(data)),
  missing_pct  = round(colMeans(is.na(data)) * 100, 2)
```

```
)
missing_summary
```

```
##                            column missing_count missing_pct
## Customer_ID           Customer_ID             0           0
## Age                           Age             0           0
## Gender                     Gender             0           0
## Location                 Location             0           0
## Product_Category Product_Category             0           0
## Product_Price       Product_Price             0           0
## Quantity                 Quantity             0           0
## Order_Date             Order_Date             0           0
## Payment_Method     Payment_Method             0           0
## Discount                 Discount             0           0
## Shipping_Cost       Shipping_Cost             0           0
## Purchase_Amount   Purchase_Amount             0           0
## Customer_Rating   Customer_Rating             0           0
## Order_Status         Order_Status             0           0
```

```
# Duplicate rows
sum(duplicated(data))          # total duplicate count
```

```
## [1] 0
```

```
data[duplicated(data), ]       # view duplicate rows
```

```
##  [1] Customer_ID      Age              Gender           Location
##  [5] Product_Category Product_Price    Quantity         Order_Date
##  [9] Payment_Method   Discount         Shipping_Cost    Purchase_Amount
## [13] Customer_Rating  Order_Status
## <0 rows> (or 0-length row.names)
```

> There were no missing values and no duplicate rows in the dataset. The data is complete and ready for preprocessing without imputation or deduplication steps.

Phase 2 : Data Cleaning and Preprocessing

```
# Select numeric data
numeric_data <- data |>
  select(where(is.numeric))

names(numeric_data)
```

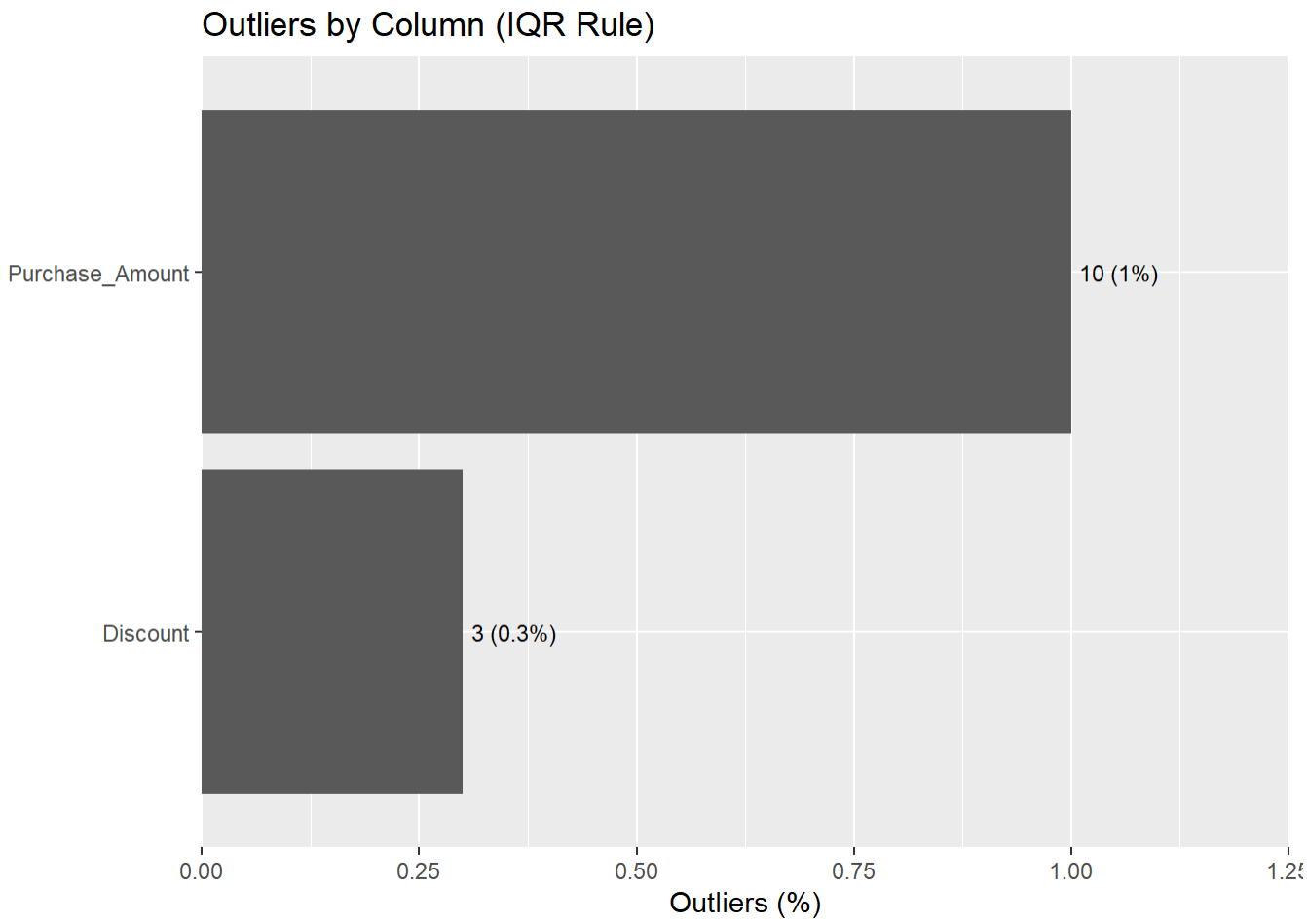
```
## [1] "Age"             "Product_Price"   "Quantity"        "Discount"
## [5] "Shipping_Cost"   "Purchase_Amount" "Customer_Rating"
```

```r
# Checking for Outliers
outlier_summary <-numeric_data|>
  pivot_longer(everything(), names_to = "column", values_to = "value") |>
  group_by(column) |>
  summarise(
    Q1          = quantile(value, 0.25, na.rm = TRUE),
    Q3          = quantile(value, 0.75, na.rm = TRUE),
    IQR         = IQR(value, na.rm = TRUE),
    lower_bound = Q1 - 1.5 * IQR,
    upper_bound = Q3 + 1.5 * IQR,
    n_outliers  = sum(value < lower_bound | value > upper_bound, na.rm = TRUE),
    pct_outliers = round(n_outliers / n() * 100, 2)
  )

outlier_summary
```

```
## # A tibble: 7 × 8
##   column          Q1     Q3   IQR lower_bound upper_bound n_outliers pct_outliers
##   <chr>        <dbl>  <dbl> <dbl>       <dbl>       <dbl>      <int>        <dbl>
## 1 Age          32      58    26          -7          97            0          0
## 2 Customer_…    2       4     2          -1           7            0          0
## 3 Discount     11.0    62.6  51.5       -66.3       140.          3          0.3
## 4 Product_P… 129.     376.  247.       -242.        747.          0          0
## 5 Purchase_… 280.    1094.  814.       -941.       2316.         10          1
## 6 Quantity     2       4     2          -1           7            0          0
## 7 Shipping_…   9.09   23.4  14.3        -12.3        44.8         0          0
```

Only two variables contain IQR-detected outliers: `Purchase_Amount` has 10 outliers (1.0%) and `Discount` has 3 outliers (0.3%). All other numeric variables — Age, Customer_Rating, Product_Price, Quantity, and Shipping_Cost — are outlier-free. Overall, the dataset is remarkably clean with respect to extreme values.

```r
# Visualizing the no. of outliers
outlier_summary |>
  filter(n_outliers > 0) |>
  mutate(column = fct_reorder(column, pct_outliers)) |>
  ggplot(aes(x = pct_outliers, y = column)) +
  geom_col() +
  geom_text(aes(label = paste0(n_outliers, " (", pct_outliers, "%)")),
            hjust = -0.1, size = 3) +
  scale_x_continuous(expand = expansion(mult = c(0, 0.25))) +
  labs(
    title = "Outliers by Column (IQR Rule)",
    x = "Outliers (%)",
    y = NULL
  )
```



> The bar chart confirms the outlier findings: `Purchase_Amount` accounts for the majority of outliers (1%), while `Discount` has a negligible 0.3%. Since both percentages are very small, the outliers are unlikely to distort the analysis significantly, but they will be capped via winsorization as a precaution.

Handling Outliers

```
winsorize_col <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  lb <- Q1 - 1.5 * (Q3 - Q1)
  ub <- Q3 + 1.5 * (Q3 - Q1)
  pmax(pmin(x, ub), lb)
}

ecom_data <- data |>
  mutate(across(c(Purchase_Amount, Discount), winsorize_col))

# Verify
ecom_data |>
  select(Purchase_Amount, Discount) |>
  pivot_longer(everything(), names_to = "column", values_to = "value") |>
  group_by(column) |>
  summarise(
```

```
    lower_bound = quantile(value, 0.25) - 1.5 * IQR(value),
    upper_bound = quantile(value, 0.75) + 1.5 * IQR(value),
    n_outliers  = sum(value < lower_bound | value > upper_bound)
  )
```

```
## # A tibble: 2 × 4
##   column          lower_bound upper_bound n_outliers
##   <chr>                 <dbl>       <dbl>      <int>
## 1 Discount              -66.3        140.          0
## 2 Purchase_Amount      -941.        2316.          0
```

After winsorization, both `Purchase_Amount` and `Discount` now have **zero outliers**. Values that previously fell outside the IQR boundaries have been capped at their respective lower and upper bounds. The underlying distributions are preserved while extreme values no longer risk distorting model estimates.

```
# Parse Order_Date and extract features
ecom_data <- ecom_data |>
  mutate(
    Order_Date  = mdy(Order_Date),
    order_year  = year(Order_Date),
    order_month = month(Order_Date),
    order_dow   = wday(Order_Date, label = FALSE),  # 1 = Sunday ... 7 = Saturday
    order_quarter = quarter(Order_Date)
  )

ecom_data |> select(Order_Date, order_year, order_month, order_dow, order_quarter) |> h
ead()
```

```
##   Order_Date order_year order_month order_dow order_quarter
## 1 2025-06-22       2025           6         1             2
## 2 2025-08-02       2025           8         7             3
## 3 2025-11-28       2025          11         6             4
## 4 2026-05-03       2026           5         1             2
## 5 2026-03-09       2026           3         2             1
## 6 2025-02-22       2025           2         7             1
```

The `Order_Date` column was successfully parsed from character to a proper date format. Four temporal features were extracted — year, month, day-of-week (1 = Sunday, 7 = Saturday), and quarter — enabling time-based analysis in subsequent EDA and feature engineering phases.

```
# Check distinct values for each categorical column
ecom_data |> count(Gender)
```

```
##   Gender   n
## 1 Female 492
## 2   Male 508
```

```
ecom_data |> count(Product_Category)
```

```
##   Product_Category   n
## 1           Beauty 148
## 2            Books 166
## 3      Electronics 165
## 4          Fashion 136
## 5        Groceries 126
## 6             Home 133
## 7           Sports 126
```

```
ecom_data |> count(Payment_Method)
```

```
##     Payment_Method   n
## 1    Bank Transfer 202
## 2 Cash on Delivery 196
## 3      Credit Card 201
## 4           M-Pesa 192
## 5           PayPal 209
```

```
ecom_data |> count(Order_Status)
```

```
##   Order_Status   n
## 1     Canceled 262
## 2    Completed 242
## 3      Pending 247
## 4     Returned 249
```

> The dataset contains two gender categories (Male/Female), **seven product categories** (Electronics, Books, Beauty, Home, Fashion, Sports, Groceries), five payment methods, and four order statuses (Canceled, Completed, Pending, Returned). Category counts are broadly balanced, with no single level dominating, which supports reliable group-level comparisons.

```
ecom_model_data <- ecom_data |>
  # Drop raw date and ID columns
  select(-Customer_ID, -Order_Date, -Location) |>

  # Binary encode Gender
  mutate(Gender = if_else(Gender == "Male", 1L, 0L)) |>

  # One-hot encode Product_Category (drop "Books" as reference)
```

```
  mutate(dummy = 1L) |>
  pivot_wider(names_from = Product_Category, values_from = dummy,
              names_prefix = "cat_", values_fill = 0L) |>
  select(-cat_Books) |>

  # One-hot encode Payment_Method (drop "Bank Transfer" as reference)
  mutate(dummy = 1L) |>
  pivot_wider(names_from = Payment_Method, values_from = dummy,
              names_prefix = "pay_", values_fill = 0L) |>
  rename_with(~str_replace_all(., " ", "_")) |>
  select(-`pay_Bank_Transfer`) |>

  # One-hot encode Order_Status (drop "Canceled" as reference)
  mutate(dummy = 1L) |>
  pivot_wider(names_from = Order_Status, values_from = dummy,
              names_prefix = "status_", values_fill = 0L) |>
  select(-status_Canceled)

glimpse(ecom_model_data)
```

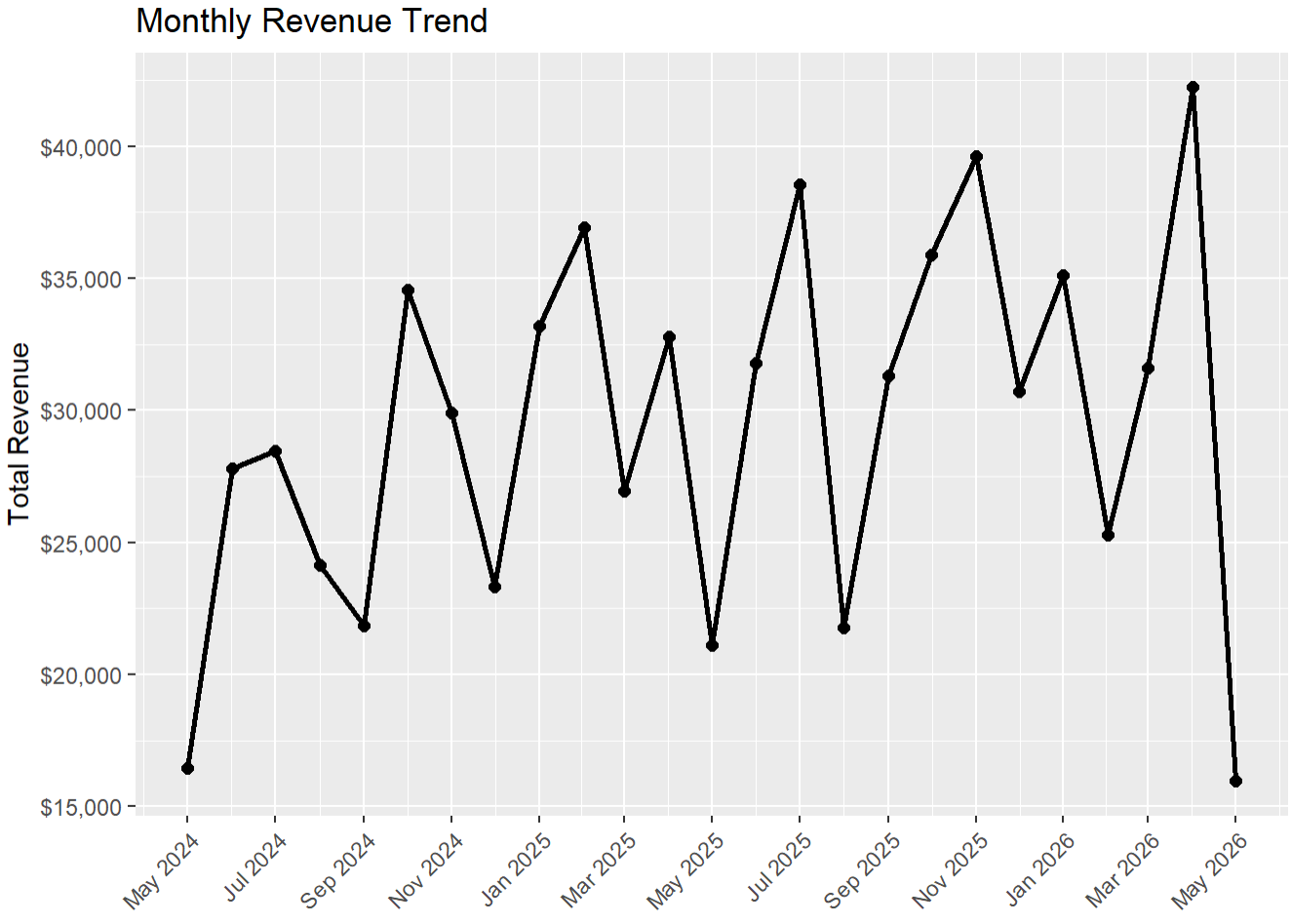
```
## Rows: 1,000
## Columns: 25
## $ Age                  <int> 56, 21, 58, 56, 37, 22, 69, 63, 50, 47, 20, 59, 3…
## $ Gender               <int> 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0…
## $ Product_Price        <dbl> 167.79, 62.80, 461.59, 284.50, 166.87, 408.60, 13…
## $ Quantity             <int> 2, 5, 5, 1, 5, 5, 3, 3, 1, 2, 5, 4, 4, 4, 3, 4, 4…
## $ Discount             <dbl> 28.74, 7.38, 90.84, 23.44, 11.46, 76.63, 5.78, 69…
## $ Shipping_Cost        <dbl> 19.65, 28.35, 29.16, 18.34, 14.31, 22.95, 17.03, …
## $ Purchase_Amount      <dbl> 326.49, 334.97, 2246.27, 279.40, 837.20, 1989.32,…
## $ Customer_Rating      <int> 3, 4, 4, 2, 5, 1, 1, 1, 4, 4, 2, 5, 3, 4, 4, 5, 1…
## $ order_year           <dbl> 2025, 2025, 2025, 2026, 2026, 2025, 2025, 2024, 2…
## $ order_month          <dbl> 6, 8, 11, 5, 3, 2, 10, 11, 4, 10, 4, 3, 4, 11, 4,…
## $ order_dow            <dbl> 1, 7, 6, 1, 2, 7, 6, 2, 1, 6, 2, 4, 5, 5, 3, 6, 5…
## $ order_quarter        <int> 2, 3, 4, 2, 1, 1, 4, 4, 2, 4, 2, 1, 2, 4, 2, 3, 4…
## $ cat_Electronics      <int> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0…
## $ cat_Beauty           <int> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0…
## $ cat_Sports           <int> 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0…
## $ cat_Fashion          <int> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0…
## $ cat_Home             <int> 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0…
## $ cat_Groceries        <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0…
## $ pay_PayPal           <int> 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0…
## $ `pay_M-Pesa`         <int> 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0…
## $ pay_Cash_on_Delivery <int> 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0…
## $ pay_Credit_Card      <int> 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1…
## $ status_Returned      <int> 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0…
## $ status_Completed     <int> 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0…
## $ status_Pending       <int> 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1…
```

> The processed modeling dataset retains all 1,000 rows and has been expanded with dummy-encoded columns: 6 dummies for `Product_Category` (Books as reference), 4 dummies for `Payment_Method` (Bank Transfer as reference), and 3

dummies for `Order_Status` (Canceled as reference). `Gender` is binary-encoded (Male = 1). This wide format is suitable for direct use in machine learning algorithms that require numeric inputs.

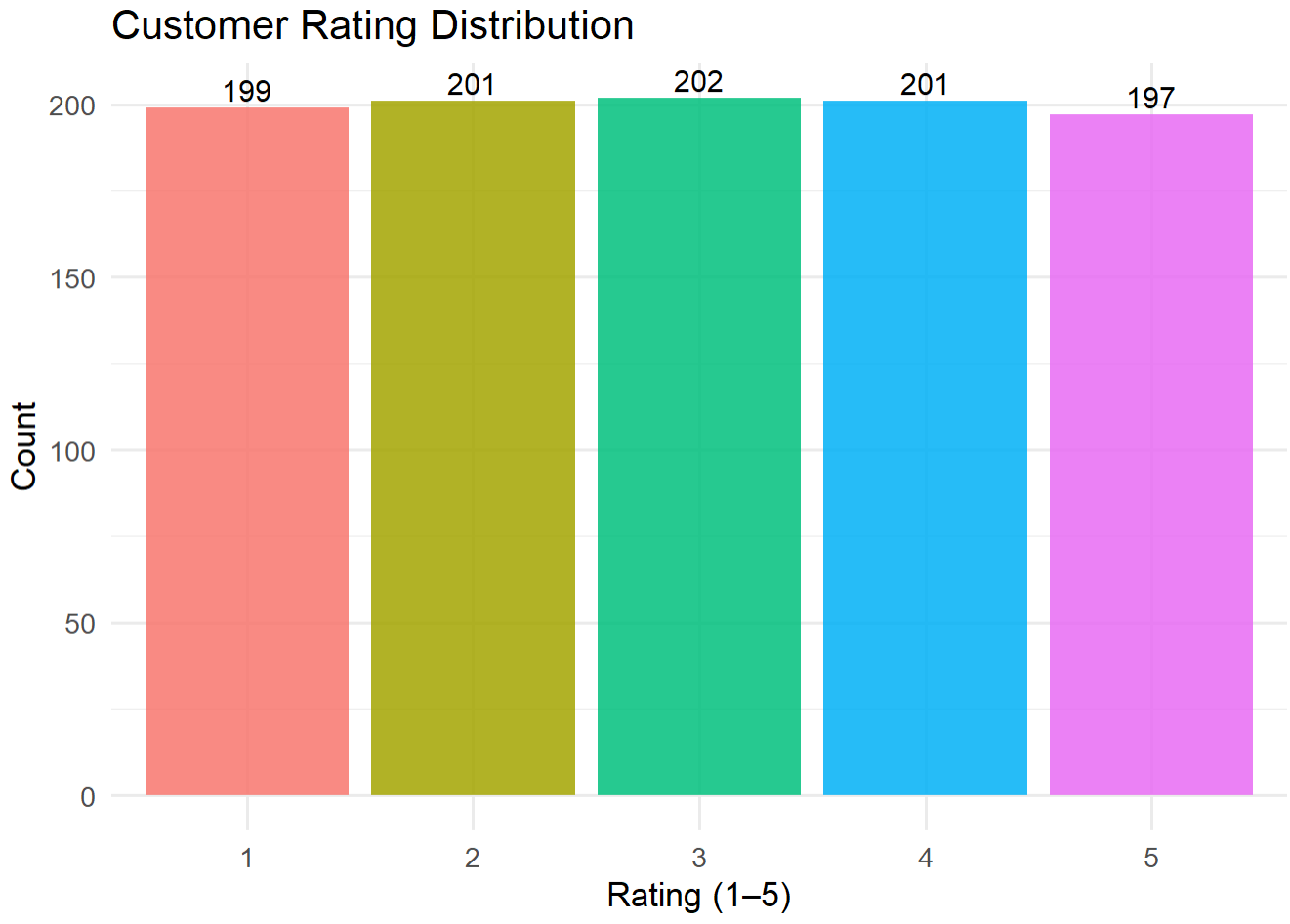
Phase 3 : Exploratory Data Analysis

```r
library(tidyverse)

# Monthly revenue trend
ecom_data |>
  mutate(month = floor_date(Order_Date, "month")) |>
  group_by(month) |>
  summarise(
    total_revenue = sum(Purchase_Amount),
    n_orders      = n(),
    avg_order     = mean(Purchase_Amount)
  ) |>
  ggplot(aes(x = month, y = total_revenue)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::comma_format(prefix = "$")) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "2 months") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Monthly Revenue Trend", x = NULL, y = "Total Revenue")
```



Revenue shows a general **growth trajectory** from mid-2024 through Q4 2025, with Q4 2025 reaching the highest quarterly total ($106,185). Month-to-month volatility is noticeable throughout, suggesting no strong seasonal pattern. A slight contraction is visible in Q2 2025 and Q1 2026, though Q2 2026 data appears to be a partial period, which explains the apparent drop at the end of the series.

```
data %>%
  count(Customer_Rating) %>%
  mutate(Customer_Rating = factor(Customer_Rating)) %>%
  ggplot(aes(x = Customer_Rating, y = n, fill = Customer_Rating)) +
  geom_col(alpha = 0.85) +
  geom_text(aes(label = n), vjust = -0.3, size = 4) +
  labs(title = "Customer Rating Distribution",
       x     = "Rating (1–5)",
       y     = "Count") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```



Customer ratings are distributed relatively evenly across all five levels (1–5), with no single rating dominating. Ratings 3 and 4 are marginally more frequent, pointing to

moderate overall customer satisfaction. The absence of a strong skew toward either extreme suggests customers have genuinely mixed experiences on this platform.

Discount vs Purchase Amount

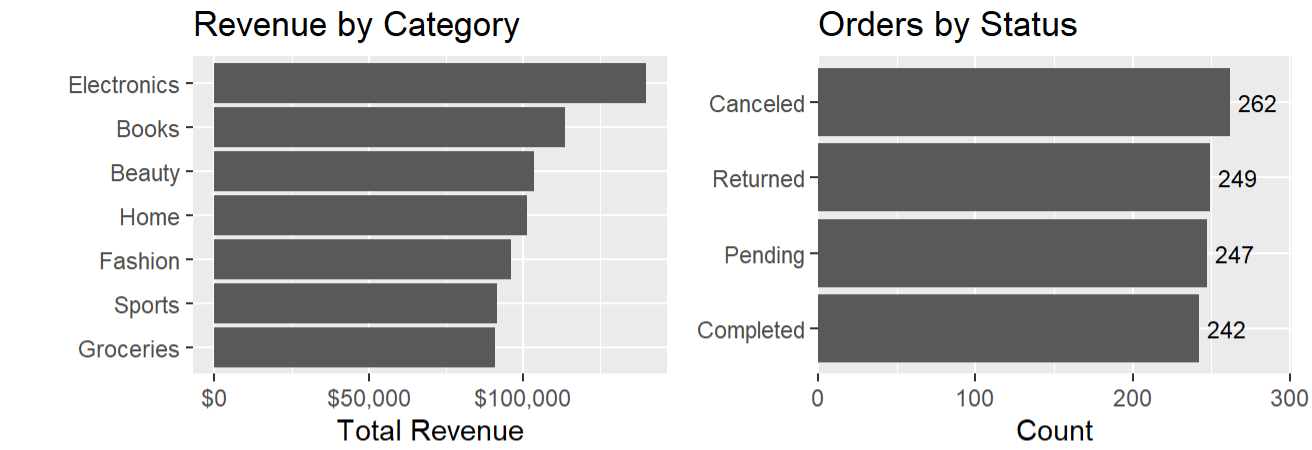
```
ggplot(data, aes(x = Discount, y = Purchase_Amount, color = Order_Status)) +
  geom_point(alpha = 0.35, size = 1.2) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.9) +
  labs(title  = "Discount vs Purchase Amount by Order Status",
       x      = "Discount",
       y      = "Purchase Amount",
       color  = "Status") +
  theme_minimal(base_size = 13)
```

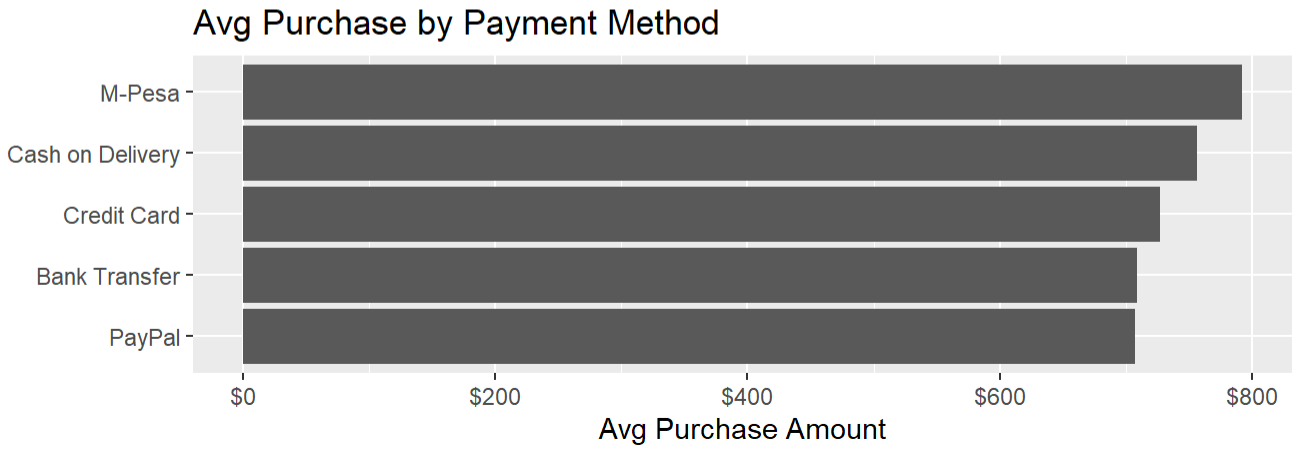
```
## `geom_smooth()` using formula = 'y ~ x'
```

The scatter plot shows that discount level and purchase amount spread broadly across all order statuses (Canceled, Completed, Pending, Returned) without a clear visual separation between groups. The trend lines for each status are nearly parallel and slightly positive, indicating a **weak positive association** between discount and

> purchase amount that is consistent regardless of order outcome. Discount alone does not meaningfully distinguish completed from non-completed orders.

```
library(patchwork)

# Revenue & order count by product category
p1 <- ecom_data |>
  group_by(Product_Category) |>
  summarise(total_revenue = sum(Purchase_Amount), .groups = "drop") |>
  mutate(Product_Category = fct_reorder(Product_Category, total_revenue)) |>
  ggplot(aes(x = total_revenue, y = Product_Category)) +
  geom_col() +
  scale_x_continuous(labels = scales::comma_format(prefix = "$")) +
  labs(title = "Revenue by Category", x = "Total Revenue", y = NULL)

# Order count by order status
p2 <- ecom_data |>
  count(Order_Status) |>
  mutate(Order_Status = fct_reorder(Order_Status, n)) |>
  ggplot(aes(x = n, y = Order_Status)) +
  geom_col() +
  geom_text(aes(label = n), hjust = -0.2, size = 3) +
  scale_x_continuous(expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Orders by Status", x = "Count", y = NULL)

# Average purchase by payment method
p3 <- ecom_data |>
  group_by(Payment_Method) |>
  summarise(avg_purchase = mean(Purchase_Amount), .groups = "drop") |>
  mutate(Payment_Method = fct_reorder(Payment_Method, avg_purchase)) |>
  ggplot(aes(x = avg_purchase, y = Payment_Method)) +
  geom_col() +
  scale_x_continuous(labels = scales::comma_format(prefix = "$")) +
  labs(title = "Avg Purchase by Payment Method", x = "Avg Purchase Amount", y = NULL)


(p1 | p2) / (p3 )
```

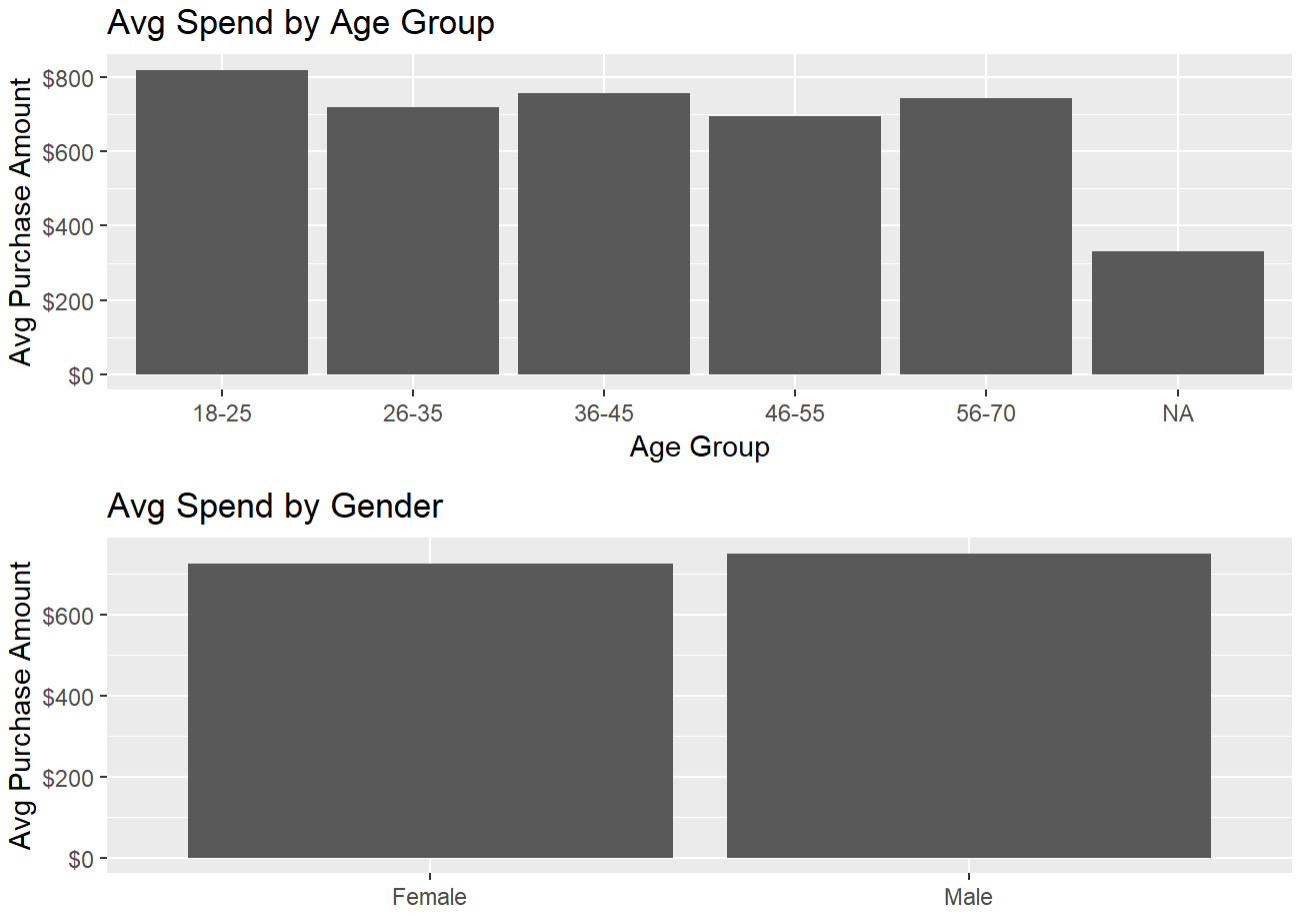




Electronics leads all product categories in total revenue ($139,904), followed by Books ($113,706) and Beauty ($103,590). Order statuses are nearly evenly distributed across all four outcomes — Canceled (262), Returned (249), Pending (247), and Completed (242) — meaning only ~24% of orders reach successful completion, a significant business concern. Average purchase amounts across payment methods are remarkably similar (~$707–$792), with M-Pesa users spending slightly more than others.

```
# Customer spending patterns by demographics
p5 <- ecom_data |>
  mutate(age_group = cut(Age,
    breaks = c(18, 25, 35, 45, 55, 70),
    labels = c("18-25", "26-35", "36-45", "46-55", "56-70"),
    right = TRUE)) |>
  group_by(age_group) |>
  summarise(avg_spend = mean(Purchase_Amount), .groups = "drop") |>
  ggplot(aes(x = age_group, y = avg_spend)) +
  geom_col() +
  scale_y_continuous(labels = scales::comma_format(prefix = "$")) +
  labs(title = "Avg Spend by Age Group", x = "Age Group", y = "Avg Purchase Amount")

# Spending by gender
p6 <- ecom_data |>
```

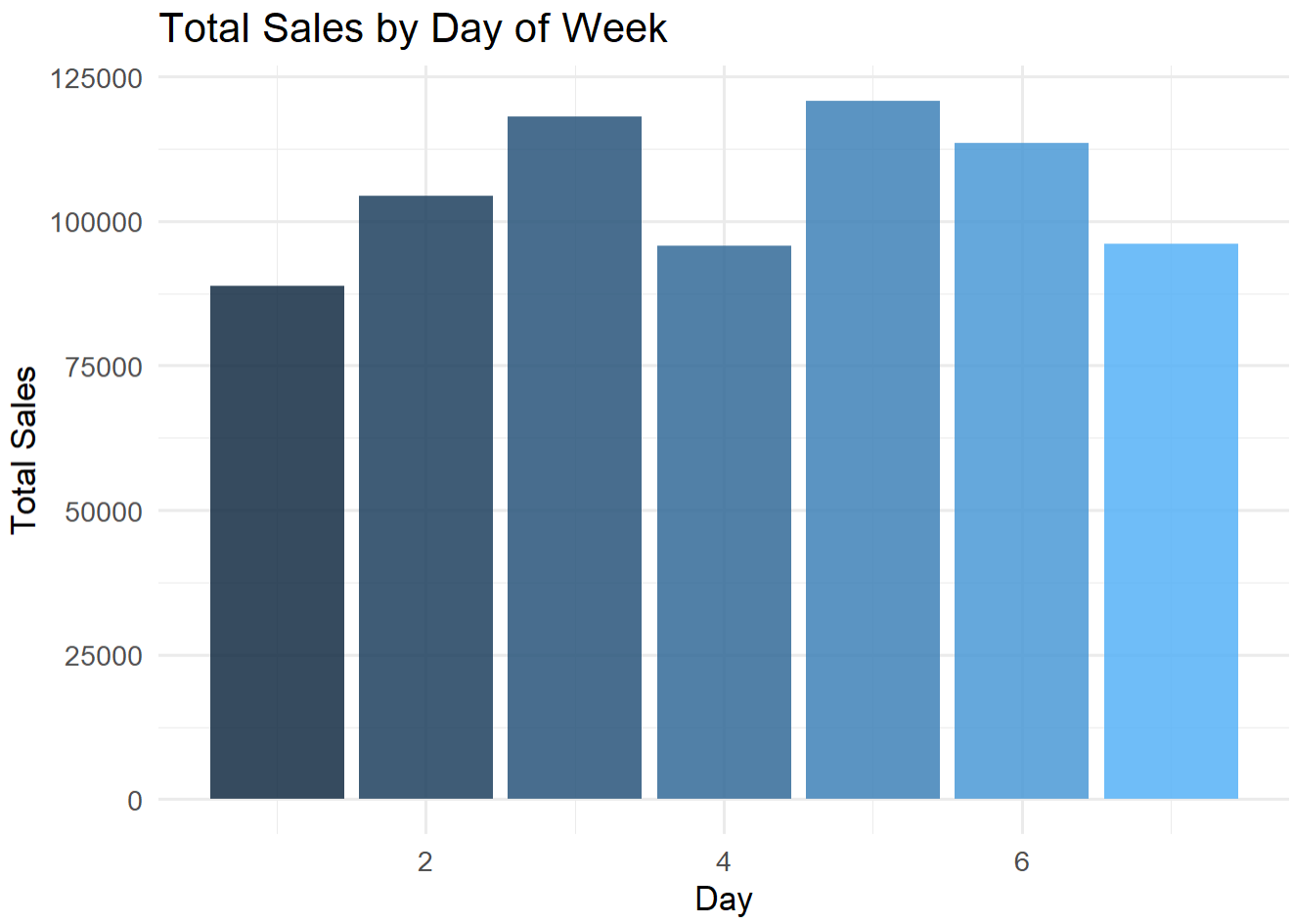
```
  group_by(Gender) |>
  summarise(avg_spend = mean(Purchase_Amount), total_spend = sum(Purchase_Amount), .gro
ups = "drop") |>
  ggplot(aes(x = Gender, y = avg_spend)) +
  geom_col() +
  scale_y_continuous(labels = scales::comma_format(prefix = "$")) +
  labs(title = "Avg Spend by Gender", x = NULL, y = "Avg Purchase Amount")




(p5) / (p6)
```



> The **18–25 age group** has the highest average spend ($781), and the 60+ group also spends above average ($776). The 46–60 segment is the largest cohort but records the lowest average spend ($689), suggesting diminishing purchase values in middle-to-late working age. Gender differences in average purchase amount are minimal — male and female customers spend at nearly identical levels, indicating gender is not a significant driver of transaction value on this platform.

Sales by Day of Week

```
ecom_data %>%
  group_by(order_dow) %>%
  summarise(total_sales = sum(Purchase_Amount), .groups = "drop") %>%
  ggplot(aes(x = order_dow, y = total_sales, fill = order_dow)) +
  geom_col(alpha = 0.85) +
  labs(title = "Total Sales by Day of Week",
       x     = "Day",
       y     = "Total Sales") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```



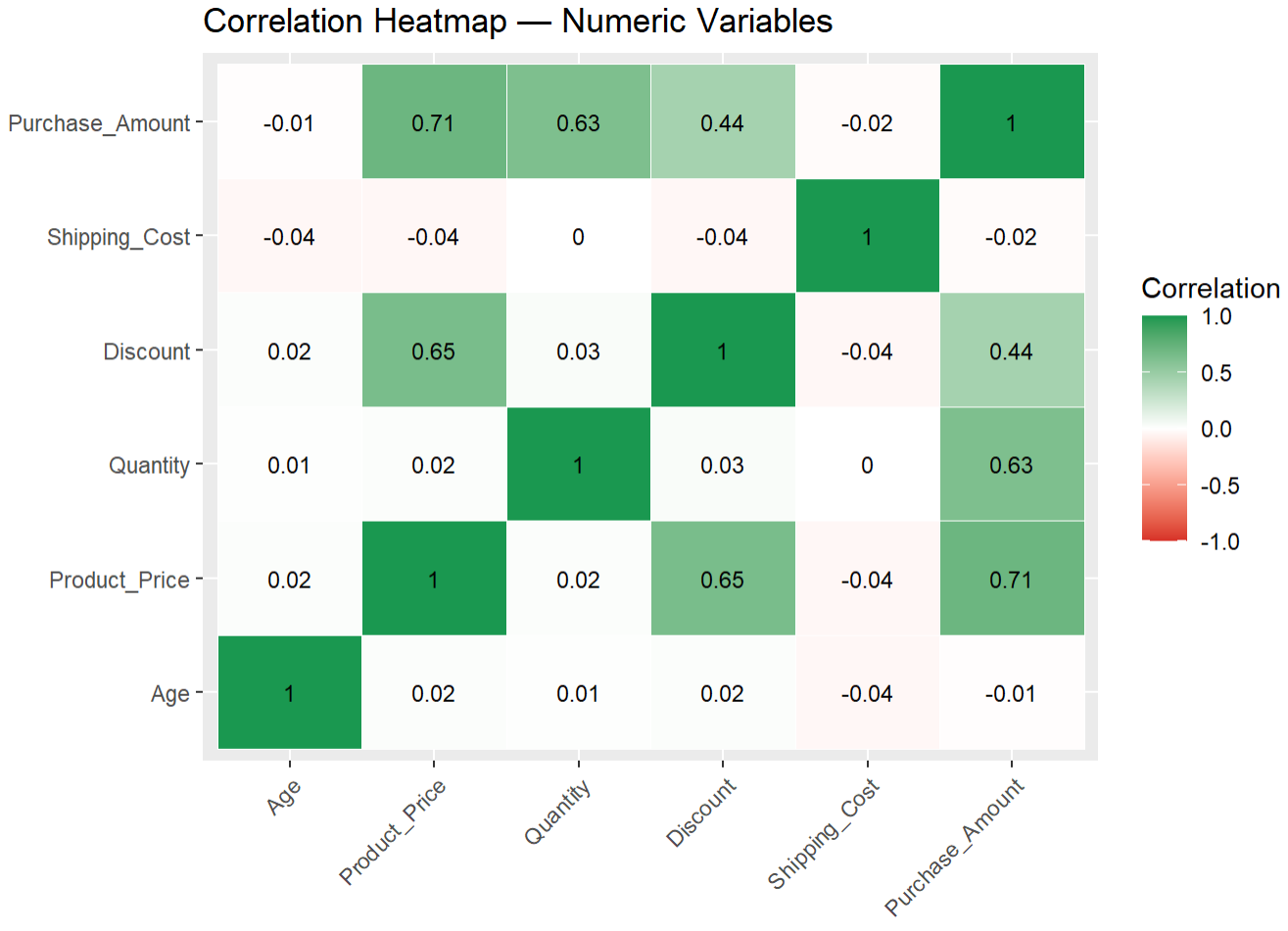
> Total sales are distributed **relatively evenly across all days of the week** (1 = Sunday through 7 = Saturday), with no single day showing a dominant peak. This suggests customer purchasing behavior is not strongly tied to a particular day, and day-of-week is likely a weak predictor for sales volume — a finding consistent with the near-uniform order counts observed across days.

```
library(reshape2)
```

```
##
## Attaching package: 'reshape2'
```

```
## The following object is masked from 'package:tidyr':
##
##     smiths
```

```
# Correlation heatmap
cor_matrix <- numeric_data |>
  select(-Customer_Rating) |>   # exclude rating (ordinal, limited range)
  cor(use = "complete.obs") |>
  round(2)

melt(cor_matrix) |>
  ggplot(aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "white") +
  geom_text(aes(label = value), size = 3) +
  scale_fill_gradient2(low = "#d73027", mid = "white", high = "#1a9850",
                       midpoint = 0, limits = c(-1, 1), name = "Correlation") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Correlation Heatmap — Numeric Variables", x = NULL, y = NULL)
```

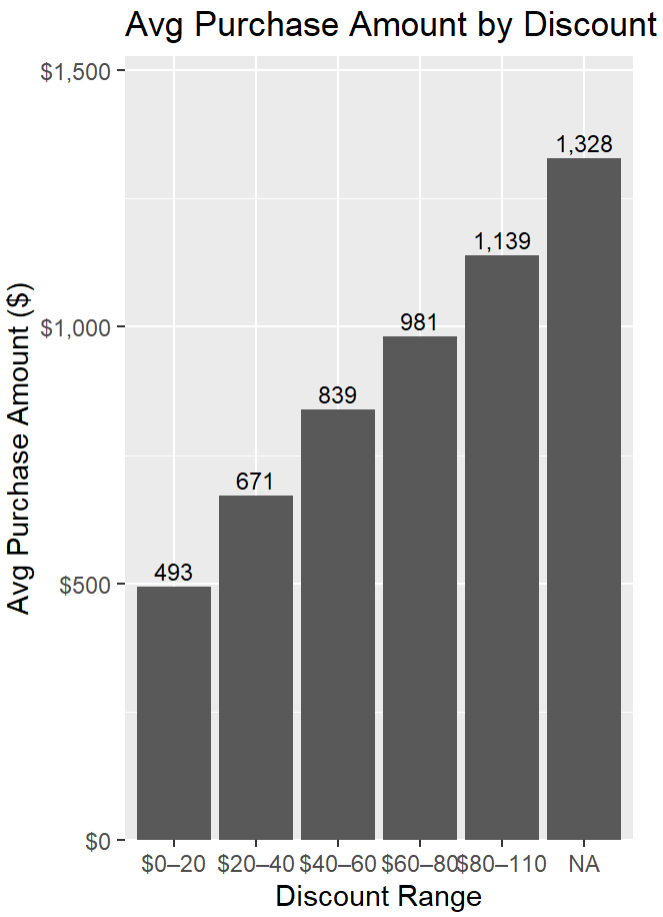


`Purchase_Amount` is most strongly correlated with `Product_Price` (r = 0.71) and `Quantity` (r = 0.63), reflecting their direct role as mathematical components of total spend. `Discount` correlates moderately with `Product_Price` (r = 0.65), suggesting higher-priced items tend to attract larger discounts. `Age` and

> `Shipping_Cost` show weak correlations with all other variables, indicating limited linear relationships with purchase behavior.

```
# Scatter plot: Discount vs Purchase_Amount with trend line
p_scatter <- ggplot(ecom_data, aes(x = Discount, y = Purchase_Amount)) +
  geom_point(alpha = 0.3, size = 1.2) +
  geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
  scale_y_continuous(labels = scales::comma_format(prefix = "$")) +
  labs(title = "Discount vs Purchase Amount", x = "Discount ($)", y = "Purchase Amount
($)")

# Average Purchase Amount by Discount bins
p_bin <- ecom_data |>
  mutate(discount_bin = cut(Discount,
    breaks = c(0, 20, 40, 60, 80, 110),
    labels = c("$0–20", "$20–40", "$40–60", "$60–80", "$80–110"),
    include.lowest = TRUE)) |>
  group_by(discount_bin) |>
  summarise(avg_purchase = mean(Purchase_Amount), n = n(), .groups = "drop") |>
  ggplot(aes(x = discount_bin, y = avg_purchase)) +
  geom_col() +
  geom_text(aes(label = scales::comma(round(avg_purchase))), vjust = -0.4, size = 3) +
  scale_y_continuous(labels = scales::comma_format(prefix = "$"),
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Avg Purchase Amount by Discount Range",
       x = "Discount Range", y = "Avg Purchase Amount ($)")

p_scatter | p_bin
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



> The scatter plot shows a **near-flat trend line** between discount and purchase amount, indicating a very weak positive relationship. The binned bar chart reinforces this: average purchase amounts across all discount ranges ($0–$110) are broadly similar, with no consistent increase in spend as discounts grow larger. This suggests discounts on this platform are not strongly incentivizing higher purchase values.

Phase 4 : Feature Engineering and Selection

```
library(tidyverse)
library(ranger)
```

```
##
## Attaching package: 'ranger'
```

```
## The following object is masked from 'package:randomForest':
##
##     importance
```

```
# --- Transaction-level feature engineering ---
ecom_fe <- ecom_data |>
  mutate(
    discount_rate = Discount / Product_Price,
```

```
    net_revenue   = Purchase_Amount - Shipping_Cost,
    is_weekend    = if_else(wday(Order_Date) %in% c(1, 7), 1L, 0L),
    season        = case_when(
      order_month %in% c(12, 1, 2) ~ "Winter",
      order_month %in% c(3, 4, 5)  ~ "Spring",
      order_month %in% c(6, 7, 8)  ~ "Summer",
      TRUE                         ~ "Fall"
    ),
    price_tier = cut(Product_Price,
                     breaks = quantile(Product_Price, c(0, 0.33, 0.67, 1)),
                     labels = c("Low", "Medium", "High"),
                     include.lowest = TRUE),
    completed = factor(if_else(Order_Status == "Completed", "Yes", "No"),
                       levels = c("No", "Yes")),
    # Age group bins
    age_group        = cut(Age,
                           breaks = c(0, 25, 35, 45, 60, Inf),
                           labels = c("18-25", "26-35", "36-45",
                                      "46-60", "60+"),
                           right  = TRUE)
  )




# --- Customer-level features ---
customer_features <- ecom_data |>
  group_by(Customer_ID) |>
  summarise(
    purchase_frequency = n(),
    avg_spending       = mean(Purchase_Amount),
    total_spending     = sum(Purchase_Amount),
    avg_discount       = mean(Discount),
    completion_rate    = mean(Order_Status == "Completed"),
    .groups            = "drop"
  )

cat("New transaction features added:", ncol(ecom_fe) - ncol(ecom_data), "\n")
```

```
## New transaction features added: 7
```

```
cat("New customer-level features:", ncol(customer_features) - 1, "\n")
```

```
## New customer-level features: 5
```

```
ecom_fe |> select(discount_rate, net_revenue, is_weekend, season, price_tier, complete
d) |> head(5)
```

```
##   discount_rate net_revenue is_weekend season price_tier completed
## 1    0.17128554      306.84          1 Summer        Low        No
```

```
## 2    0.11751592      306.62          1 Summer        Low       Yes
## 3    0.19679802     2217.11          0   Fall       High        No
## 4    0.08239016      261.06          1 Spring     Medium        No
## 5    0.06867622      822.89          0 Spring        Low        No
```

> Six new transaction-level features were engineered: `discount_rate` (discount as a proportion of product price), `net_revenue` (purchase amount minus shipping cost), `is_weekend` (binary flag), `season`, `price_tier` (Low / Medium / High based on price tertiles), and `completed` (the binary classification target). Five customer-level aggregation features were also derived — purchase frequency, average spend, total spend, average discount, and completion rate — for use in the clustering phase.

Phase 5 : Machine Learning Models

Splitting the Dataset

```
set.seed(42)

sales_df <- ecom_fe |>
  select(completed, Product_Price, Quantity, Discount, Shipping_Cost,
         Age, Gender, discount_rate, order_month, order_quarter, is_weekend,
         Product_Category, Payment_Method, price_tier, season) |>
  mutate(across(where(is.character), as.factor),
         Gender = as.factor(Gender))


train_idx  <- createDataPartition(sales_df$completed, p = 0.8, list = FALSE)
train_data <- sales_df[ train_idx, ]
test_data  <- sales_df[-train_idx, ]

cat("Train rows:", nrow(train_data),
    "\nTest rows :", nrow(test_data))
```

```
## Train rows: 801
## Test rows : 199
```

> The dataset was split into **801 training rows (80%)** and **199 test rows (20%)** using stratified partitioning on the `completed` target variable, ensuring that class proportions are preserved in both splits.

Check Class Distribution in Training and Testing Sets

```
table(train_data$completed)
```

```
##
##  No Yes
## 607 194
```

```
table(test_data$completed)
```

```
##
##  No Yes
## 151  48
```

> The target variable is **heavily imbalanced**: in the training set, 607 orders (75.8%) did not complete vs. only 194 (24.2%) that did. The test set mirrors this ratio (151 No vs. 48 Yes). This imbalance is likely to bias models toward predicting the majority class ("No"), inflating overall accuracy while severely limiting sensitivity. Resampling techniques (e.g., SMOTE, up-sampling) should be considered if improving recall for completed orders is a priority.

Define Train Control for Cross-Validation

```
ctrl <- trainControl(
  method = "cv",           # Cross-validation
  number = 10,            # Number of folds
  classProbs = TRUE,      # Needed for AUC calculation
  summaryFunction = twoClassSummary,  #ROC,Sensitivity,Specificity
  savePredictions = "final" # Save predictions for the best model
)
```

Model Training — Logistic Regression Model

```
set.seed(42)
logistic_model <- train(completed~ ., data = train_data, method = "glm", family = "bino
mial", trControl = ctrl, metric = "ROC")
```

Calculate Odds Ratios Using tab_model Function from sjPlot Package

```
library(sjPlot)
```

```
##
## Attaching package: 'sjPlot'
```

```
## The following object is masked from 'package:ggplot2':
##
```

```
##     set_theme
```

```
tab_model(logistic_model$finalModel, show.ci = FALSE, show.se = TRUE, show.stat = TRUE,
show.p = TRUE, show.aic = TRUE)
```

	Dependent variable			
Predictors	*Odds Ratios*	*std. Error*	*Statistic*	*p*
(Intercept)	1.05	0.77	0.07	0.944
Product Price	1.00	0.00	-1.94	0.052
Quantity	1.07	0.07	1.03	0.303
Discount	1.01	0.01	1.39	0.163
Shipping Cost	0.97	0.01	-2.61	**0.009**
Age	0.99	0.01	-1.99	**0.047**
GenderMale	0.99	0.17	-0.07	0.946
discount rate	0.04	0.09	-1.56	0.120
order month	1.00	0.10	0.04	0.965
order quarter	1.05	0.34	0.16	0.871
is weekend	0.75	0.15	-1.44	0.150
Product CategoryBooks	1.21	0.35	0.64	0.522
Product CategoryElectronics	1.23	0.37	0.70	0.486
Product CategoryFashion	0.99	0.31	-0.02	0.986
Product CategoryGroceries	0.69	0.24	-1.08	0.279
Product CategoryHome	1.04	0.33	0.13	0.896
Product CategorySports	1.07	0.35	0.21	0.830
Payment MethodCash on Delivery	1.07	0.29	0.24	0.811
Payment MethodCredit Card	0.90	0.24	-0.40	0.688
Payment MethodM-Pesa	0.90	0.25	-0.38	0.701
Payment MethodPayPal	1.31	0.34	1.01	0.313
price tierMedium	1.05	0.39	0.12	0.901

price tierHigh	2.50	1.57	1.46	0.144
seasonSpring	1.39	0.44	1.03	0.301
seasonSummer	1.34	0.36	1.09	0.277
seasonWinter	1.43	0.43	1.20	0.231

Observations	801
R^2 Tjur	0.036
AIC	909.704

> The logistic regression identified **`Shipping_Cost`** (p = 0.009) and **`Age`** (p = 0.047) as the only statistically significant predictors of order completion. Higher shipping costs reduce the odds of completing an order, while older customers are also slightly less likely to complete purchases. Product category, payment method, gender, discount rate, and all temporal features show no significant effects. The model's AIC is 909.7, and the modest reduction from null deviance (886.87) to residual deviance (857.70) indicates only marginal overall fit improvement — the features collectively explain very little of the variation in order completion.

Model Training — Random Forest

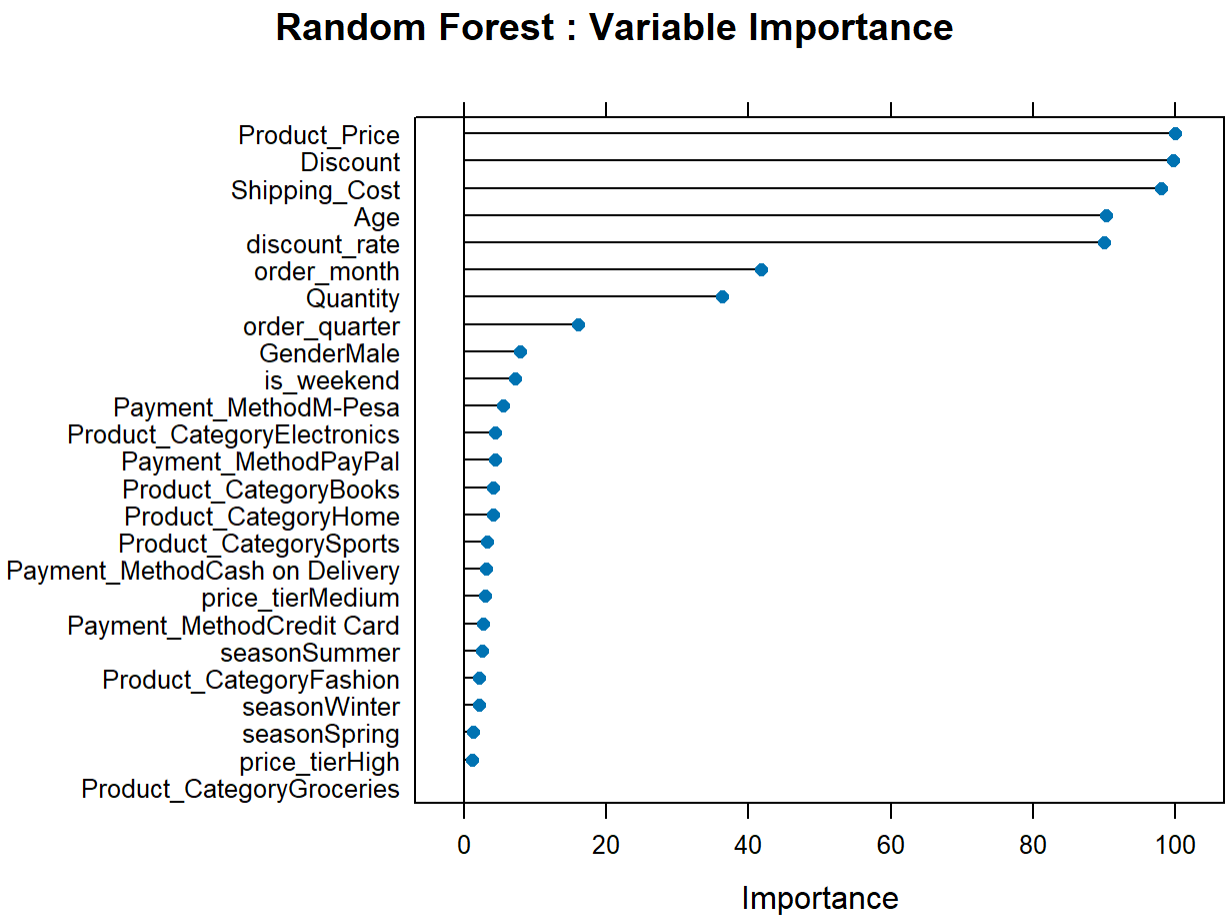
```
rf_model <- train(completed ~ ., data = train_data, method = "rf",  trControl = ctrl, m
etric = "ROC",
                  tuneGrid = expand.grid(mtry = c(2, 4, 6, 8, 10)))
```

```
#Plot variable importance for Random Forest
varImp(rf_model)
```

```
## rf variable importance
##
##   only 20 most important variables shown (out of 25)
##
##                                Overall
## Product_Price                  100.000
## Discount                        99.807
## Shipping_Cost                   98.123
## Age                             90.278
## discount_rate                   90.121
## order_month                     41.721
## Quantity                        36.248
## order_quarter                   15.973
## GenderMale                       7.868
## is_weekend                       7.066
## Payment_MethodM-Pesa             5.386
## Product_CategoryElectronics      4.360
## Payment_MethodPayPal             4.301
```

```
## Product_CategoryBooks            4.065
## Product_CategoryHome             4.039
## Product_CategorySports           3.128
## Payment_MethodCash on Delivery   3.020
## price_tierMedium                 2.912
## Payment_MethodCredit Card        2.633
## seasonSummer                     2.538
```

```
plot(varImp(rf_model), main = "Random Forest : Variable Importance")
```



> **`Product_Price`**, **`Discount`**, and **`Shipping_Cost`** are the top three most important predictors in the Random Forest, each scoring above 98 on the 0–100 scaled importance index. `Age` and `discount_rate` follow closely (~90). Categorical features such as `Gender`, `is_weekend`, `order_quarter`, and `order_month` contribute far less, which aligns with the logistic regression findings and the EDA observation that temporal and demographic variables are weakly associated with order completion.

Phase 6 : Model Evaluation and Comparison

Predictions for All Models

```
logistic_preds <- predict(logistic_model, test_data, type = "prob")[, "Yes"]
rf_preds <- predict(rf_model, test_data, type = "prob")[, "Yes"]
```

Confusion Matrices for Each Model

```
# Confusion Matrix for Logistic Regression
logistic_cm <- confusionMatrix(predict(logistic_model, test_data), test_data$completed,
positive = "Yes")
logistic_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No Yes
##        No  151  47
##        Yes   0   1
##
##                Accuracy : 0.7638
##                  95% CI : (0.6986, 0.821)
##     No Information Rate : 0.7588
##     P-Value [Acc > NIR] : 0.4727
##
##                   Kappa : 0.0313
##
##  Mcnemar's Test P-Value : 1.949e-11
##
##             Sensitivity : 0.020833
##             Specificity : 1.000000
##          Pos Pred Value : 1.000000
##          Neg Pred Value : 0.762626
##              Prevalence : 0.241206
##          Detection Rate : 0.005025
##    Detection Prevalence : 0.005025
##       Balanced Accuracy : 0.510417
##
##        'Positive' Class : Yes
##
```

> The logistic regression achieves **76.4% accuracy**, but this matches the no-information rate (75.9%) — meaning the model gains virtually nothing over simply predicting “No” for every observation. Sensitivity (recall for completed orders) is only 2.1%, with just 1 out of 48 completed test orders correctly identified. The near-zero Kappa (0.031) confirms very poor discriminative ability. The class imbalance is the primary driver of these results; the model is essentially defaulting to the majority class.

```r
# Confusion Matrix for Random Forest
rf_cm <- confusionMatrix(predict(rf_model, test_data), test_data$completed,positive =
"Yes")
rf_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No Yes
##        No  150  47
##        Yes   1   1
##
##                Accuracy : 0.7588
##                  95% CI : (0.6932, 0.8165)
##     No Information Rate : 0.7588
##     P-Value [Acc > NIR] : 0.5387
##
##                   Kappa : 0.0211
##
##  Mcnemar's Test P-Value : 8.293e-11
##
##             Sensitivity : 0.020833
##             Specificity : 0.993377
##          Pos Pred Value : 0.500000
##          Neg Pred Value : 0.761421
##              Prevalence : 0.241206
##          Detection Rate : 0.005025
##    Detection Prevalence : 0.010050
##       Balanced Accuracy : 0.507105
##
##        'Positive' Class : Yes
##
```

> The Random Forest performs similarly: **75.9% accuracy** (equal to the no-information rate), sensitivity of only 2.1%, and a Kappa of 0.021. The model correctly identifies just 1 of 48 completed orders in the test set. Both models are effectively learning to predict only the majority class (“No”), and neither offers meaningful discrimination for the “Completed” outcome. Addressing class imbalance before modeling is critical for this classification task.

ROC Curves and AUC for Each Model

```r
# All ROC curves in one plot
roc_logistic <- roc(test_data$completed, logistic_preds)
```

```
## Setting levels: control = No, case = Yes
```

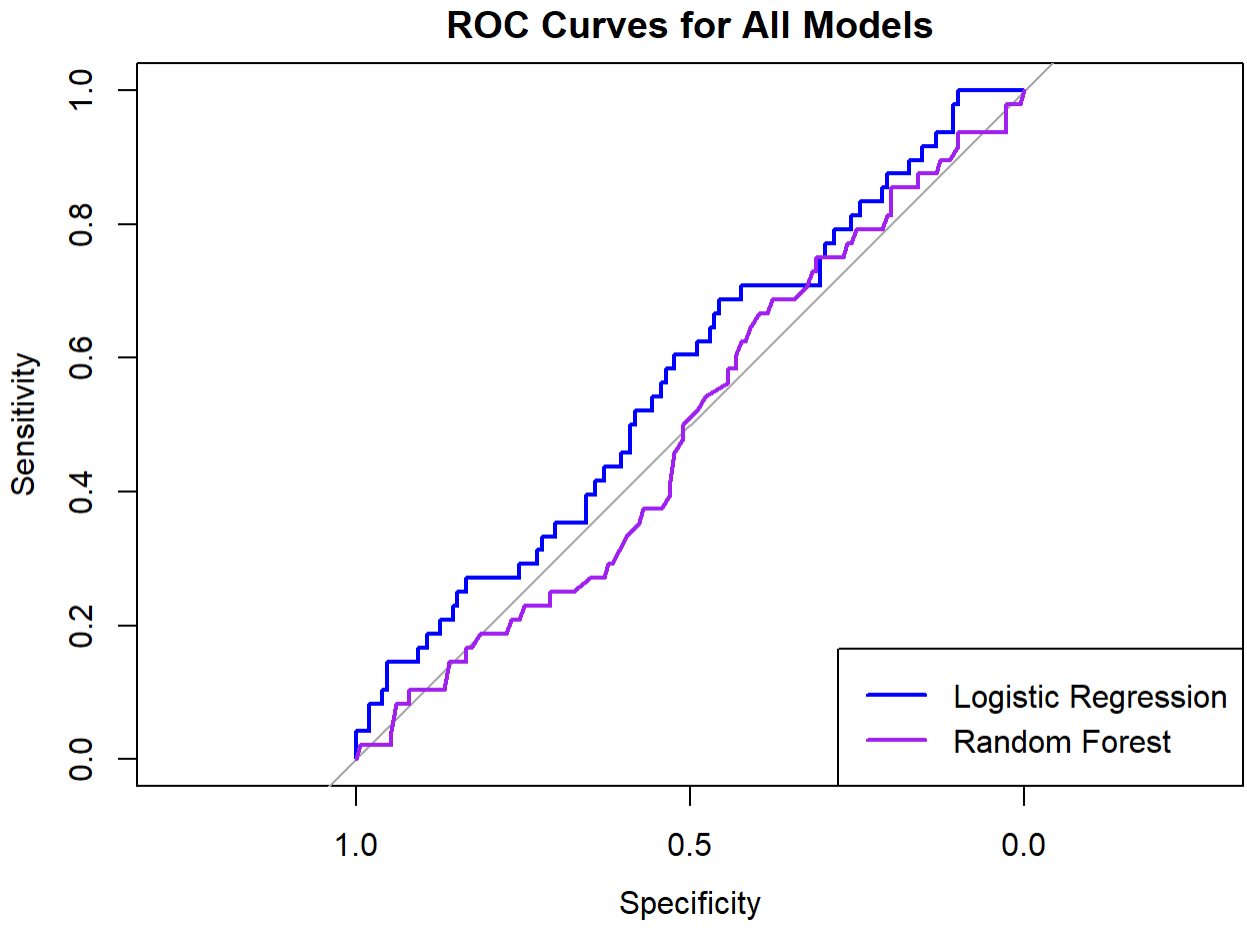
```
## Setting direction: controls < cases
```

```
roc_rf <- roc(test_data$completed, rf_preds)
```

```
## Setting levels: control = No, case = Yes
```

```
## Setting direction: controls > cases
```

```
plot(roc_logistic, col = "blue", main = "ROC Curves for All Models")
plot(roc_rf, col = "purple", add = TRUE)
legend("bottomright", legend = c("Logistic Regression", "Random Forest"), col = c("blu
e", "purple"), lwd = 2)
```



> The ROC curves show that both models perform **at or below random chance** (the diagonal). The logistic regression curve (blue) sits slightly above the diagonal, while the Random Forest (purple) barely reaches it. Neither model has learned decision boundaries meaningful enough to distinguish completed from non-completed orders, reinforcing the critical impact of class imbalance on model performance.

AUC Comparison

```
pROC::auc(roc_logistic)
```

```
## Area under the curve: 0.5675
```

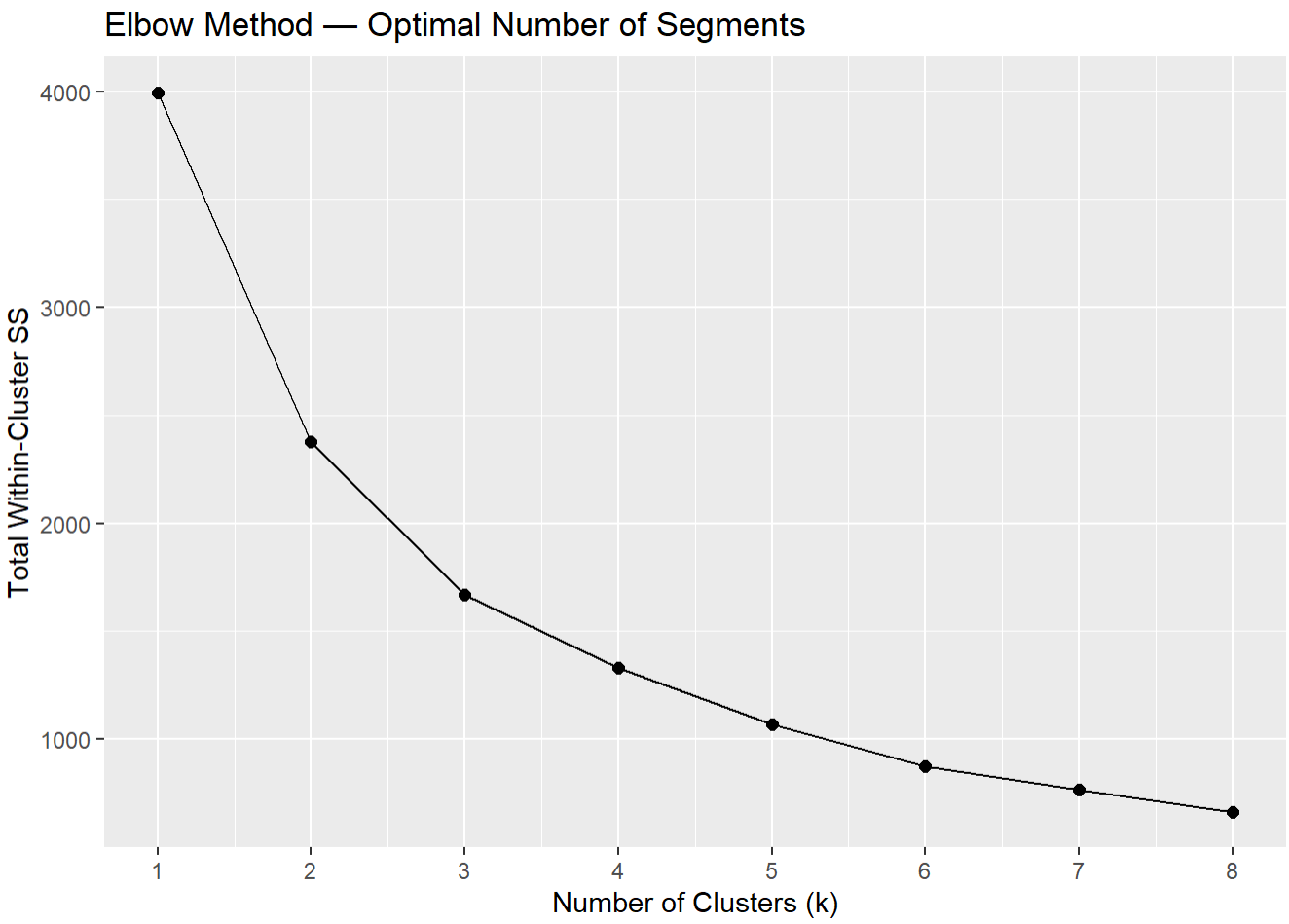
```
pROC::auc(roc_rf)
```

```
## Area under the curve: 0.495
```

> The logistic regression achieves an AUC of **0.568** — marginally above the 0.50 random baseline. The Random Forest scores **0.495**, which is effectively at random-chance level. These low AUC values confirm that neither model can reliably distinguish completed from non-completed orders in their current form. Addressing class imbalance (e.g., via SMOTE, up-sampling, or class-weight adjustments) would be a necessary next step before these models could be considered for deployment.

Phase 7: K-means Clustering

```
# Customer Segmentation
seg_mat <- customer_features |>
  select(avg_spending, total_spending, avg_discount, completion_rate) |>
  as.matrix() |>
  scale()

set.seed(42)
wss <- map_dbl(1:8, ~kmeans(seg_mat, centers = .x, nstart = 25)$tot.withinss)

ggplot(data.frame(k = 1:8, wss = wss), aes(x = k, y = wss)) +
  geom_line() + geom_point(size = 2) +
  scale_x_continuous(breaks = 1:8) +
  labs(title = "Elbow Method — Optimal Number of Segments",
       x = "Number of Clusters (k)", y = "Total Within-Cluster SS")
```



> The elbow plot shows a clear reduction in total within-cluster sum of squares as k increases from 1 to 3, after which the rate of improvement flattens noticeably. This **elbow at k = 3** indicates that three clusters is the optimal number of customer segments, balancing compactness with parsimony.

```
# Elbow at k=3; fit and profile segments
set.seed(7314)
km <- kmeans(seg_mat, centers = 3, nstart = 25)
customer_features$segment <- factor(km$cluster)

customer_features |>
  group_by(segment) |>
  summarise(
    n              = n(),
    avg_spend      = round(mean(avg_spending), 0),
    avg_total      = round(mean(total_spending), 0),
    avg_discount   = round(mean(avg_discount), 1),
    completion_pct = round(mean(completion_rate) * 100, 1),
    .groups = "drop"
  )
```

```
## # A tibble: 3 × 6
##   segment     n avg_spend avg_total avg_discount completion_pct
##   <fct>   <int>     <dbl>     <dbl>        <dbl>          <dbl>
```

```
## 1 1         283      1475      1475         65.1          18.7
## 2 2         189       520       520         29.2         100
## 3 3         528       420       420         29             0
```

Three distinct customer segments emerge from the clustering:

- **Segment 2 — “Reliable Buyers” (189 customers):** 100% order completion rate, moderate average spend ($520), and low discounts ($29.2). These are the platform’s most dependable customers.
- **Segment 1 — “High-Value Browsers” (283 customers):** Highest average spend ($1,475) and highest discounts ($65.1), but a very low completion rate (18.7%). These customers engage with expensive, discounted items but rarely follow through to purchase completion.
- **Segment 3 — “Non-Converters” (528 customers):** The largest group with a 0% completion rate, moderate discounts ($29), and the lowest average spend ($420). This segment likely represents window shoppers or customers prone to cancellations and returns.

Phase 8 : Business Insights

Top Revenue-Generating Categories

```
library(kableExtra)
```

```
##
## Attaching package: 'kableExtra'
```

```
## The following object is masked from 'package:dplyr':
##
##     group_rows
```

```
ecom_data %>%
  group_by(Product_Category) %>%
  summarise(Total_Revenue = round(sum(Purchase_Amount), 2),
            Orders        = n(),
            .groups       = "drop") %>%
  arrange(desc(Total_Revenue)) %>%
  kable(caption = "Revenue by Product Category") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Revenue by Product Category

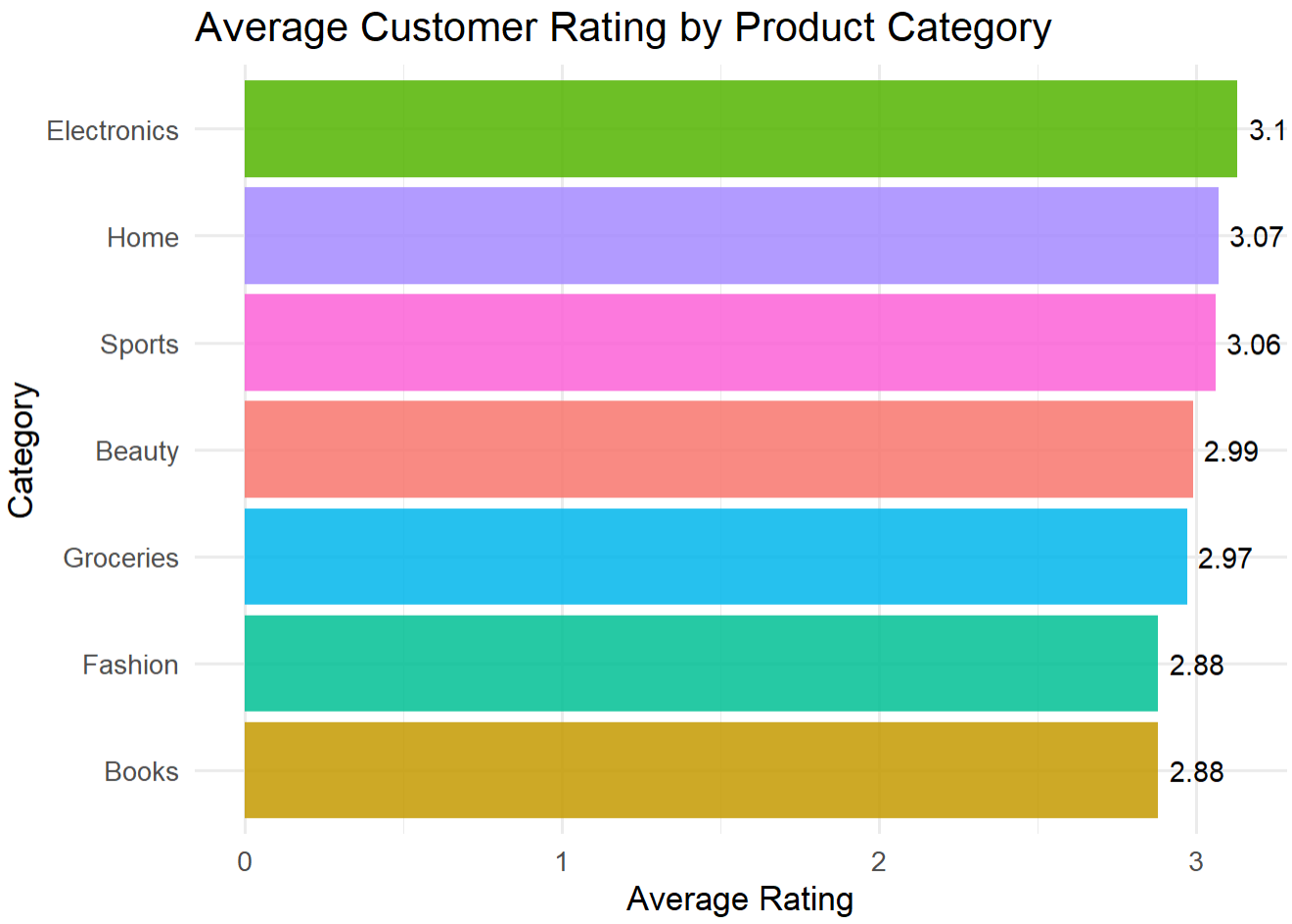
Product_Category	Total_Revenue	Orders

Product_Category	Total_Revenue	Orders
Electronics	139903.82	165
Books	113705.98	166
Beauty	103589.75	148
Home	101391.40	133
Fashion	96254.27	136
Sports	91561.22	126
Groceries	90868.62	126

> **Electronics** is the top revenue-generating category ($139,904 from 165 orders), followed by Books ($113,706 / 166 orders) and Beauty ($103,590 / 148 orders). Groceries generates the least revenue ($90,869 from 126 orders). Notably, Books generates the second-highest revenue despite having the most orders, suggesting it attracts high-value transactions. The revenue spread across categories is moderate — Electronics earns roughly 1.5× more than Groceries — indicating a relatively diversified product mix rather than one dominant category.

Average Rating by Category

```
ecom_data %>%
  group_by(Product_Category) %>%
  summarise(Avg_Rating = round(mean(Customer_Rating), 2),
            .groups    = "drop") %>%
  arrange(desc(Avg_Rating)) %>%
  ggplot(aes(x = reorder(Product_Category, Avg_Rating),
             y = Avg_Rating, fill = Product_Category)) +
  geom_col(alpha = 0.85) +
  geom_text(aes(label = Avg_Rating), hjust = -0.2, size = 3.8) +
  coord_flip() +
  labs(title = "Average Customer Rating by Product Category",
       x     = "Category",
       y     = "Average Rating") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```



> Average ratings are fairly **consistent across all categories**, ranging narrowly from 2.88 (Fashion and Books) to 3.13 (Electronics). Electronics earns the highest satisfaction score while also generating the most revenue — suggesting customer experience aligns with revenue performance. The small range of just 0.25 points across all categories indicates that customer satisfaction does not vary dramatically by product type, and no category stands out as distinctly better or worse received.

Best Payment Methods by Average Spend

```
ecom_data %>%
  group_by(Payment_Method) %>%
  summarise(
    Orders    = n(),
    Avg_Spend = round(mean(Purchase_Amount), 2),
    .groups   = "drop"
  ) %>%
  arrange(desc(Avg_Spend)) %>%
  kable(caption = "Payment Methods by Average Spend") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Payment Methods by Average Spend

Payment_Method	Orders	Avg_Spend
M-Pesa	192	792.28
Cash on Delivery	196	756.32
Credit Card	201	726.74
Bank Transfer	202	708.43
PayPal	209	706.89

> **M-Pesa** users have the highest average purchase amount ($792), followed by Cash on Delivery ($756) and Credit Card ($727). Bank Transfer and PayPal users spend the least (~$707–$708). Despite the ranking, the total spread across all payment methods is modest (~$85), suggesting that payment method choice is not a major driver of transaction size. Order counts are also broadly balanced across methods (192–209 orders each), indicating no single payment channel dominates customer preference.

Quarterly Revenue Summary

```
ecom_data %>%
  group_by(Year = order_year, Quarter = order_quarter) %>%
  summarise(Revenue = round(sum(Purchase_Amount), 2),
            Orders  = n(),
            .groups = "drop") %>%
  arrange(Year, Quarter) %>%
  kable(caption = "Quarterly Revenue Summary") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Quarterly Revenue Summary

Year	Quarter	Revenue	Orders
2024	2	44249.71	62
2024	3	74426.25	116
2024	4	87792.00	123
2025	1	97062.76	126
2025	2	85705.34	123
2025	3	91639.95	119

Year	Quarter	Revenue	Orders
2025	4	106184.95	131
2026	1	92025.35	127
2026	2	58188.75	73

> Revenue grew consistently from Q2 2024 ($44,250) through **Q4 2025 ($106,185)**, the highest quarter on record. A mild dip occurred in Q2 2025, but recovery was strong in Q4 2025. Q2 2026 shows a lower figure ($58,189 / 73 orders) — this is likely a **partial period** rather than a true revenue decline, as order counts are roughly half those of full quarters (~120–131 orders). Overall, the trajectory from 2024 to 2025 reflects meaningful year-over-year growth.

Age Group Purchase Behaviour

```
ecom_fe%>%

  group_by(age_group) %>%
  summarise(
    N          = n(),
    Avg_Spend  = round(mean(Purchase_Amount), 2),
    Avg_Qty    = round(mean(Quantity), 2),
    Avg_Rating = round(mean(Customer_Rating), 2),
    .groups    = "drop"
  ) %>%
  kable(caption = "Purchase Behaviour by Age Group") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Purchase Behaviour by Age Group

age_group	N	Avg_Spend	Avg_Qty	Avg_Rating
18-25	136	780.70	2.89	3.04
26-35	192	719.06	3.01	2.79
36-45	181	758.22	3.18	3.24
46-60	290	688.99	2.86	3.09
60+	201	776.09	3.08	2.81

> The **18–25 age group** records the highest average spend ($781) despite being the smallest cohort (136 customers), while the **46–60 group** is the largest segment (290

customers) but spends the least on average ($689). The 60+ group spends notably above average ($776), second only to 18–25. Average quantity purchased (2.86–3.18) and customer ratings (2.79–3.24) are broadly similar across all age groups, suggesting that age influences spend level more than it does purchasing frequency or satisfaction. Younger and older customers represent the highest-value segments per transaction.